



Module 1. Optimization Techniques and Gradient Descent

Loss Optimization and Gradient descent
Convergence and Training Optimization and Optimizers
Normalization
Overfitting

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a sense of depth and structure.

Loss Optimization and Gradient Descent

Loss Optimization

We want to find the network weights that achieve the **lowest loss**

$$W^* = \arg \min_W \frac{1}{n} \sum_{i=1}^n \mathcal{L} \left(f \left(x^{(i)}; W \right), y^{(i)} \right)$$

$$W^* = \arg \min_W J(W)$$

Remember: $W = \{W^{(0)}, W^{(1)}, \dots\}$

Loss Optimization

We want to find the network weights that achieve the **lowest loss**

$$W^* = \arg \min_W \frac{1}{n} \sum_{i=1}^n \mathcal{L} \left(f \left(x^{(i)}; W \right), y^{(i)} \right)$$

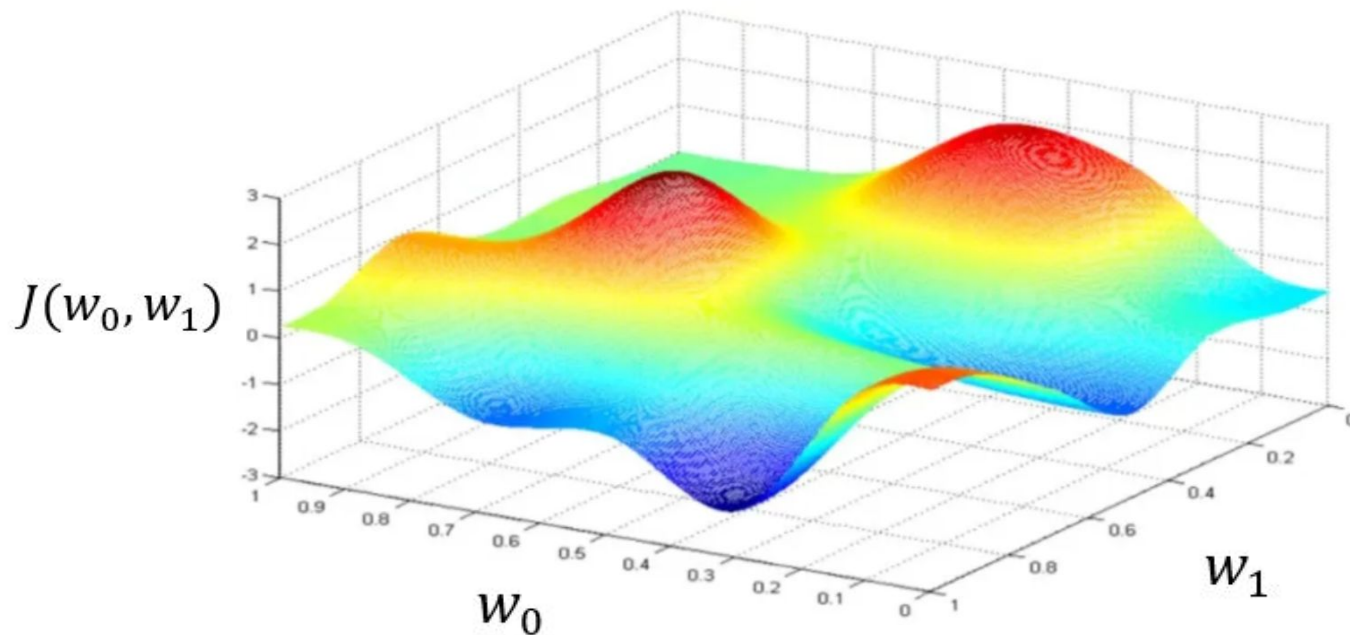
$$W^* = \arg \min_W J(W)$$

Where:

- W^* represents the optimal parameter values that minimize the loss function.
- The first equation expresses that the optimal parameters W are found by minimizing the average loss function over all training samples.
- The second equation simplifies the notation by defining the loss function as $J(W)$ indicating that the goal is to minimize this function to obtain the best model parameters.

Loss Optimization

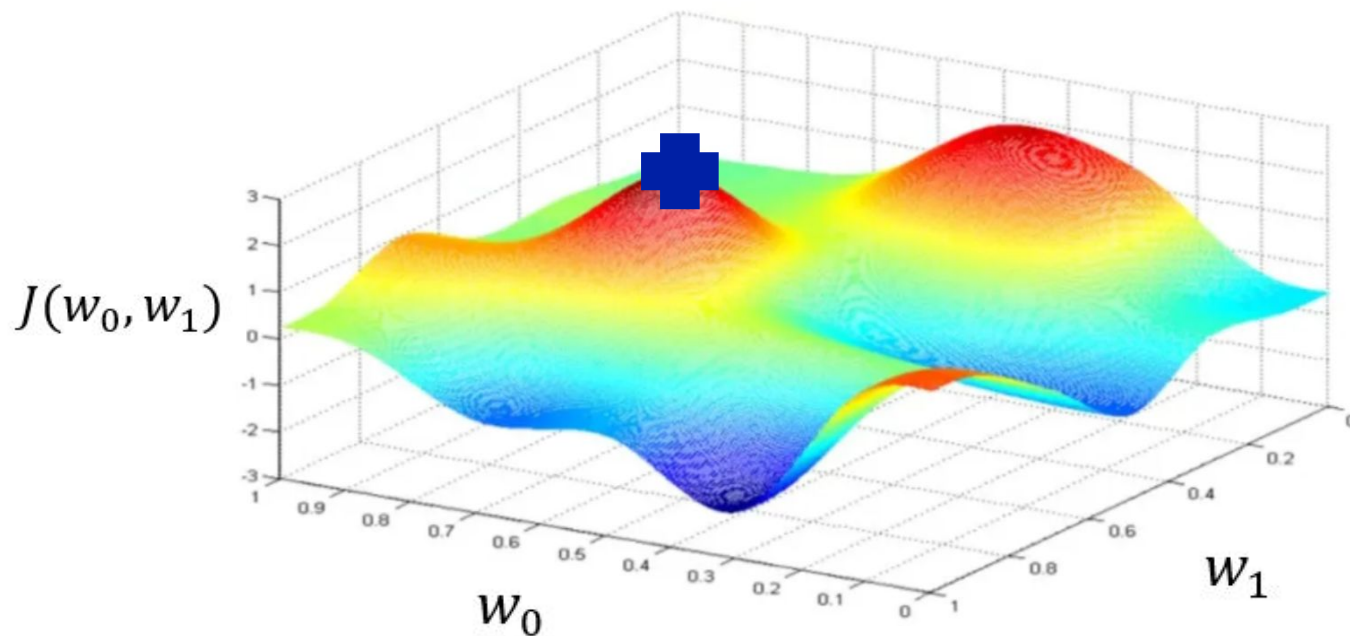
$$W^* = \arg \min_W J(W)$$



Remember:
Our loss is a
function of the
network
weights

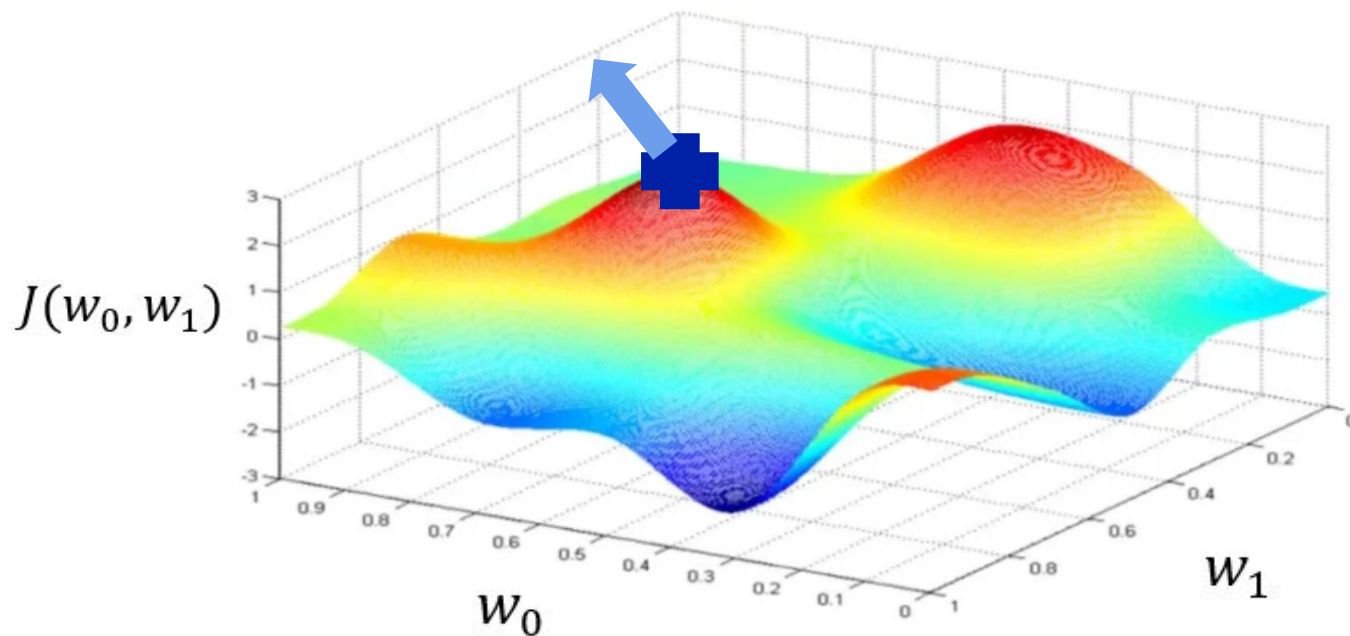
Loss Optimization

Randomly pick an initial (w_0, w_1)



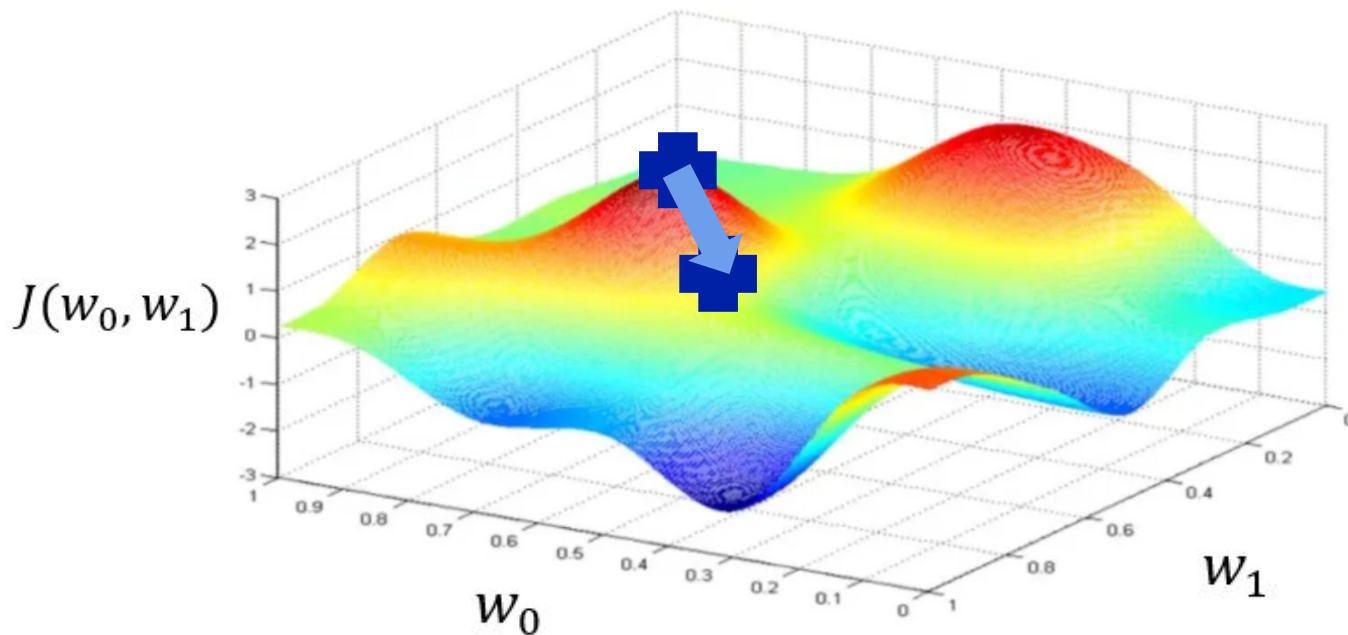
Loss Optimization

Compute gradient, $\frac{\partial J(W)}{\partial W}$



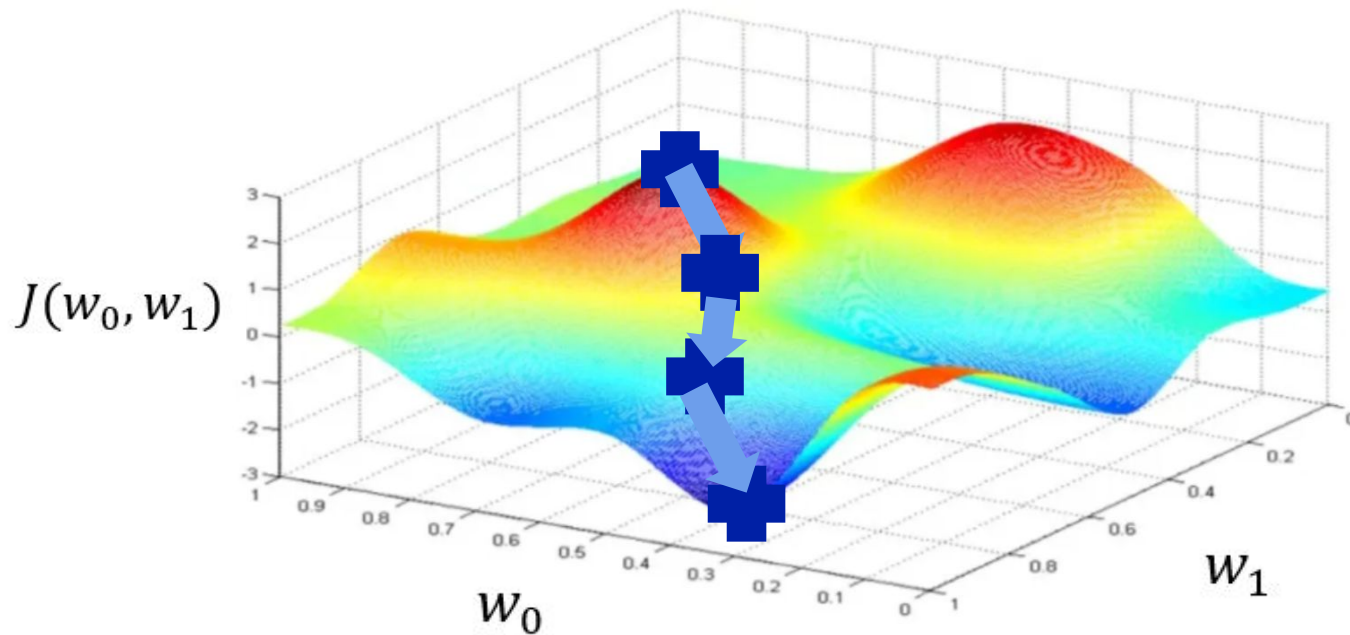
Loss Optimization

Take a small step in opposite direction of gradient



Gradient Descent

Repeat until convergence



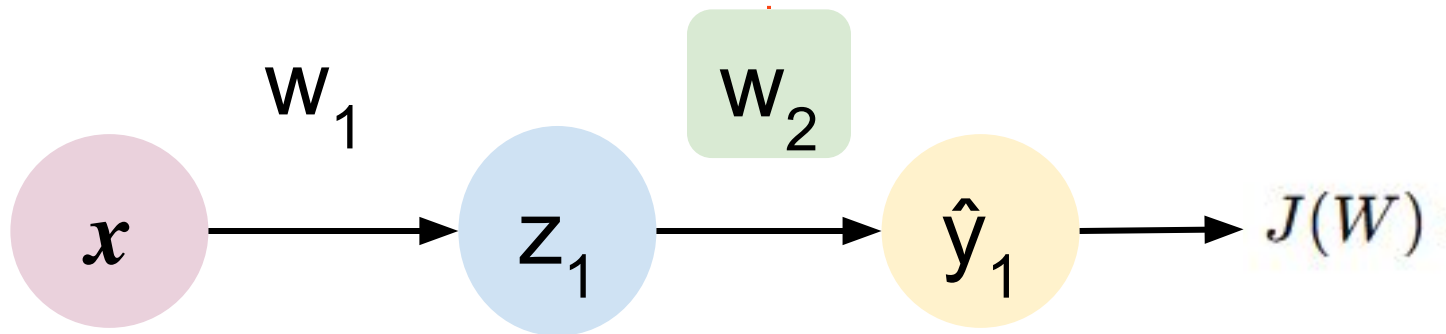
Algorithm

1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence
3. Compute gradient with respect to the weights $\frac{\partial J(W)}{\partial W}$
4. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps. The lines are of varying lengths and are arranged in a way that they seem to flow from the top left towards the bottom left.

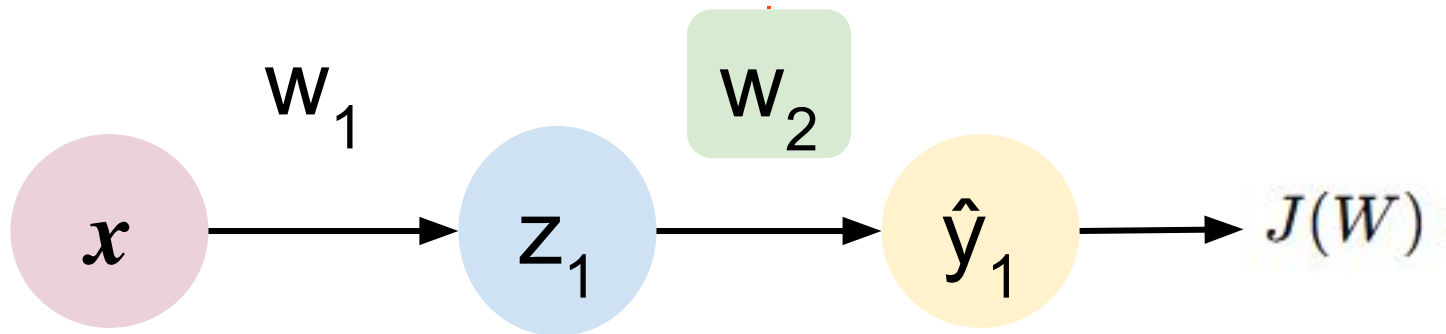
Backpropagation

Computing gradients: Backpropagation



How does a small change in one weight
(i.e. w_2), affect the final loss $J(W)$

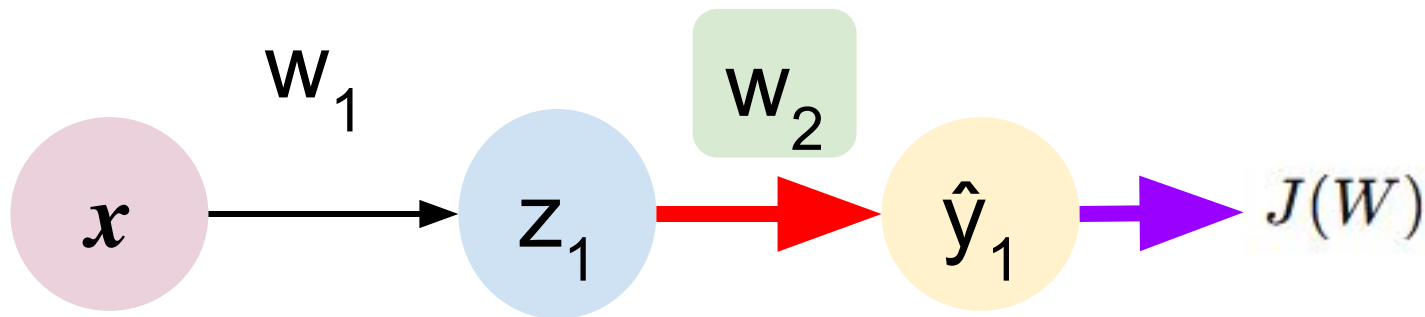
Computing gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2}$$

Let's use the chain rule

Computing gradients: Backpropagation



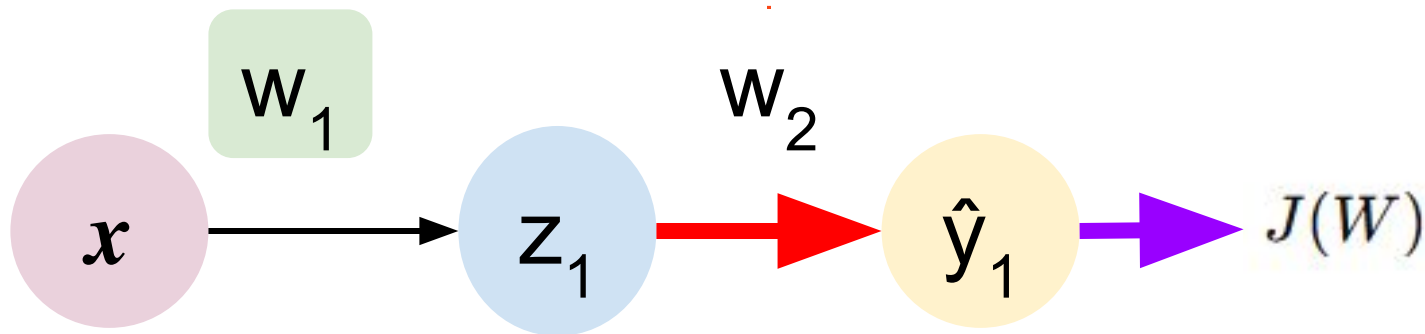
$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}$$



Apply chain
rule!

Apply chain
rule!

Computing gradients: Backpropagation

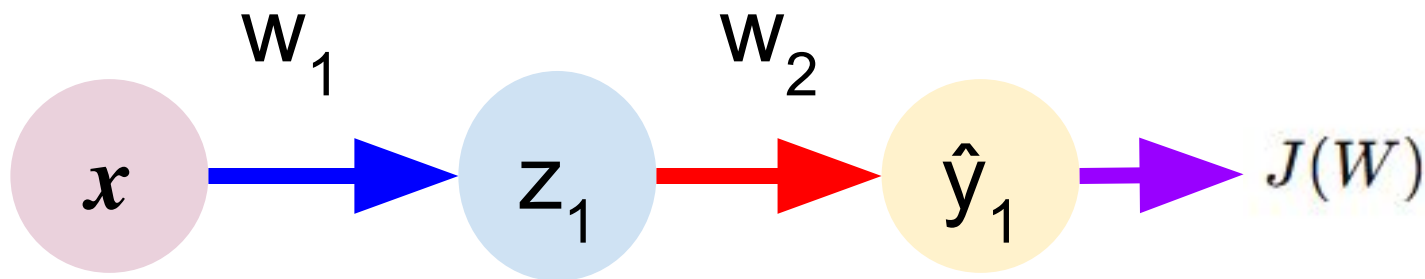


$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1}$$

Apply chain
rule!

Apply chain
rule!

Computing gradients: Backpropagation

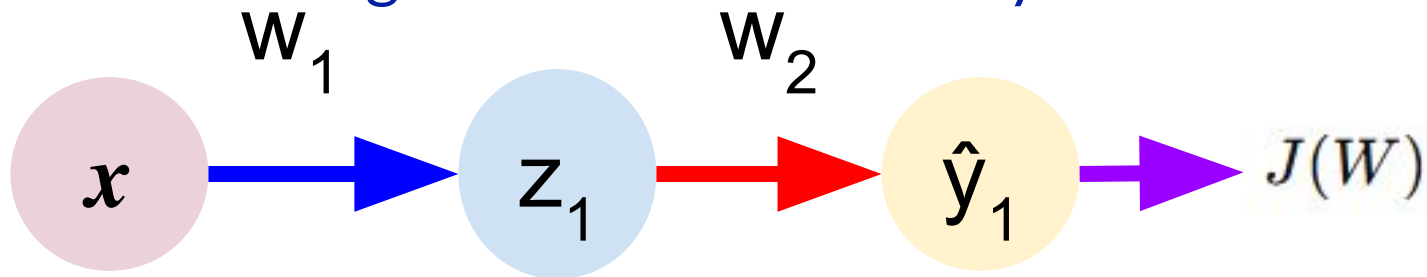


$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$



Computing gradients: Backpropagation

Repeat this for every weight in the network using gradients from later layers



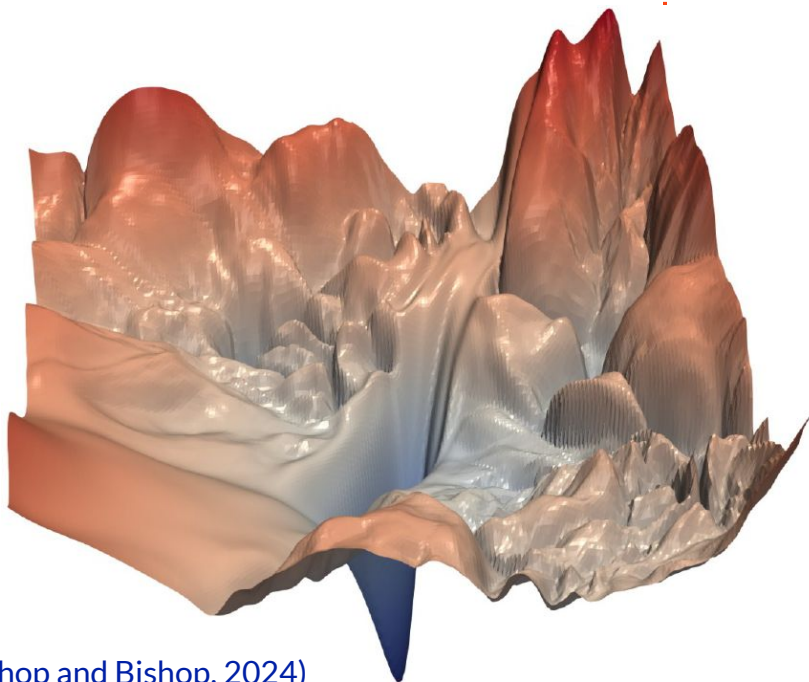
$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps. The lines are of varying lengths and are arranged in a way that they seem to flow from the top left towards the bottom left.

Convergence: Batches

Training neural networks is difficult



Error surface for a
network with 56
layers

(Bishop and Bishop, 2024)

Gradient Descent

Algorithm

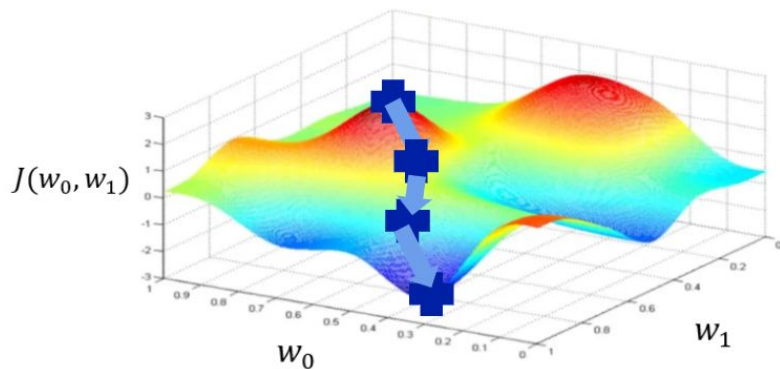
1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence
3. Compute gradient with respect to the weights $\frac{\partial J(W)}{\partial W}$
4. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

Can be very
computationally
intensive!

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
 - a. Pick single data point i
 - b. Compute gradient $\frac{\partial J_i(W)}{\partial W}$
 - c. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
3. Return weights

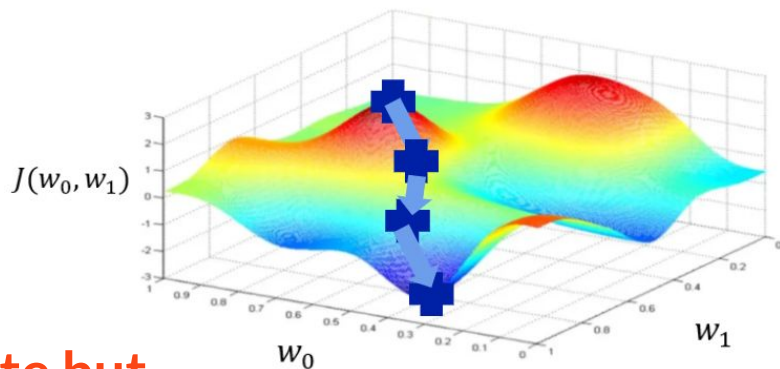


Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
 - a. Pick single data point i
 - b. Compute gradient $\frac{\partial J_i(W)}{\partial W}$
 - c. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
3. Return weights

Easy to compute but
very noisy (stochastic)



Stochastic Gradient Descent - Mini Batch

Algorithm

1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
 - a. Pick single data point i
 - b. Compute gradient $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$
 - c. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
3. Return weights

Stochastic Gradient Descent - Mini Batch

Algorithm

1. Initialize weights randomly $W \sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
 - a. Pick single data point i
 - b. Compute gradient $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$
 - c. Update weights $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
3. Return weights

Easy to compute and a much better estimate of the true gradient

Mini Batches while training

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

Mini Batches while training

Mini-batches lead to faster training!

Can parallelize computation

Achieve significant increase on GPUs

Batch Gradient Descent

- Entire dataset for updation
- Cost function reduces smoothly
- Computation cost is very high

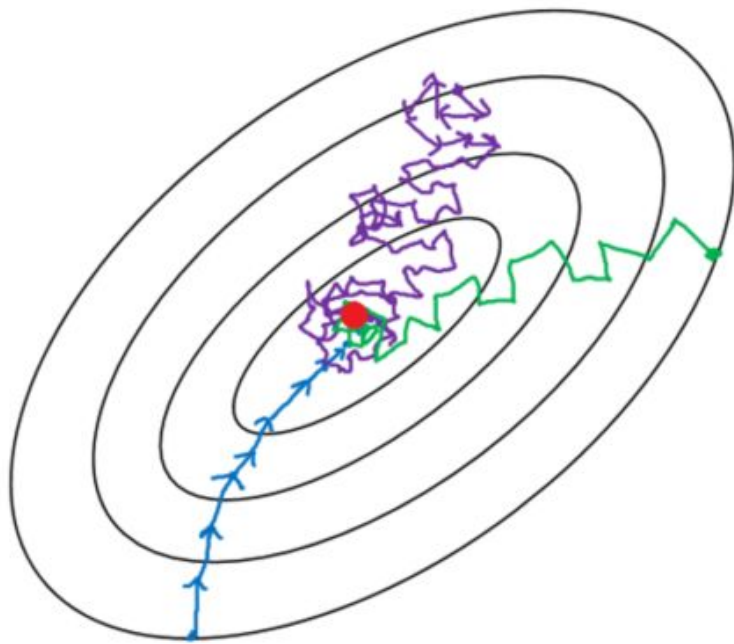
Stochastic Gradient Descent (SGD)

- Single observation for updation
- Lot of variations in cost function
- Computation time is more

Mini-Batch Gradient Descent

- Subset of data for updation
- Smoother cost function as compared to SGD
- Computation time is lesser than SGD
- Computation cost is lesser than Batch Gradient Descent

Recap: Batches



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

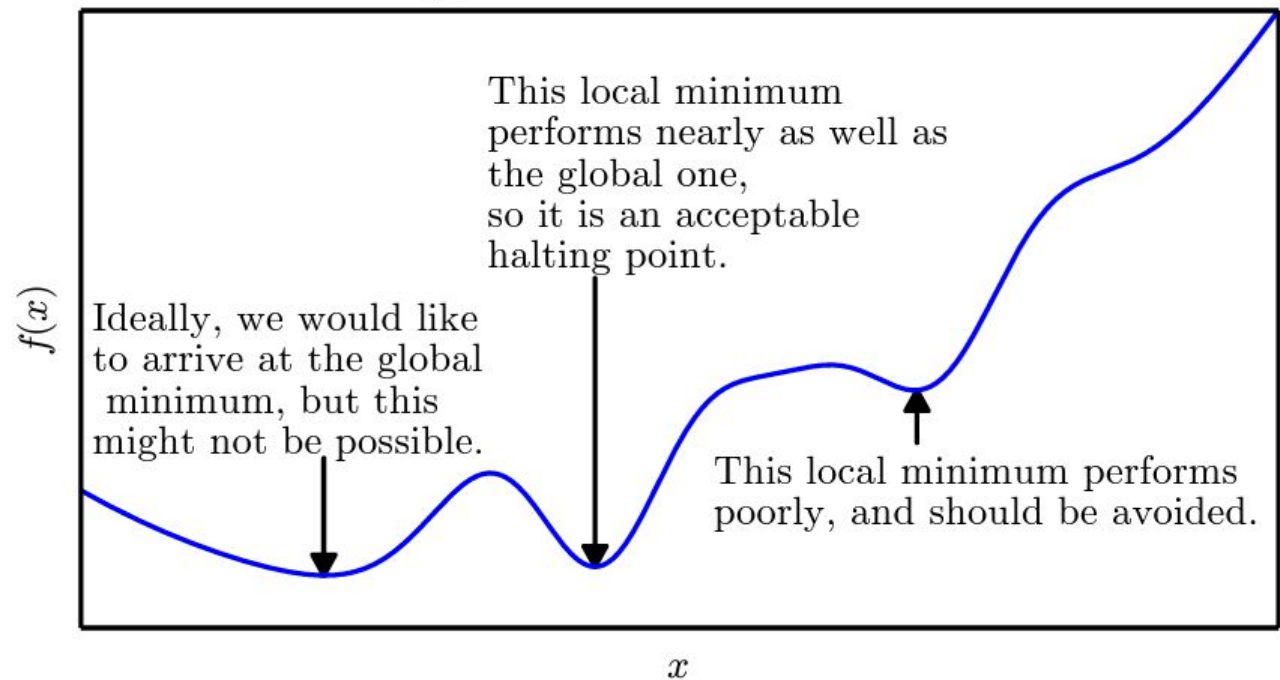
<https://sweta-nit.medium.com/batch-mini-batch-and-stochastic-gradient-descent-e9bc4cadd461>



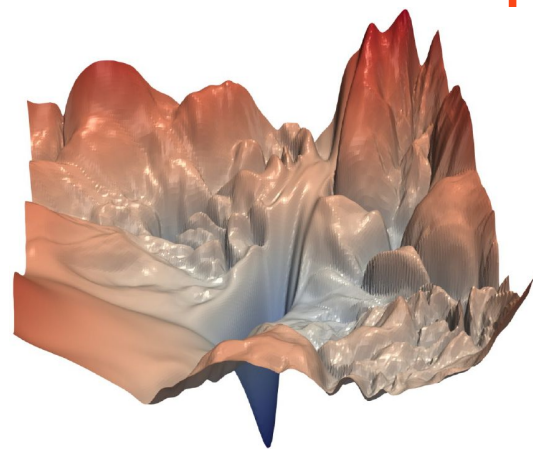
Learning Rates and Optimizers

Why Should We Care?

Approximate minimization



(Goodfellow et al., 2016)



(Bishop and Bishop, 2024)

Loss Functions can be difficult to Optimize

Remember:

Optimization through
gradient descent

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

Loss Functions can be difficult to Optimize

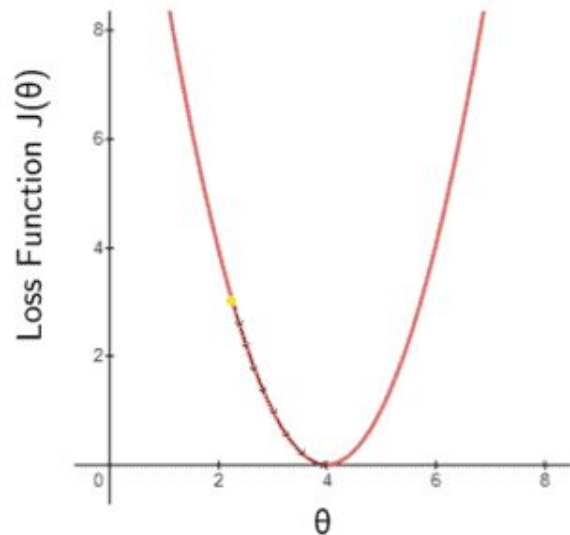
Remember:
Optimization through
gradient descent

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

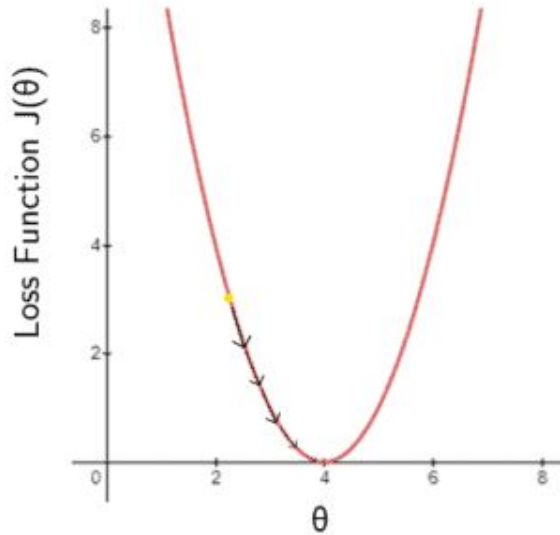
**How can we set the
learning rate?**

Setting the Learning Rate

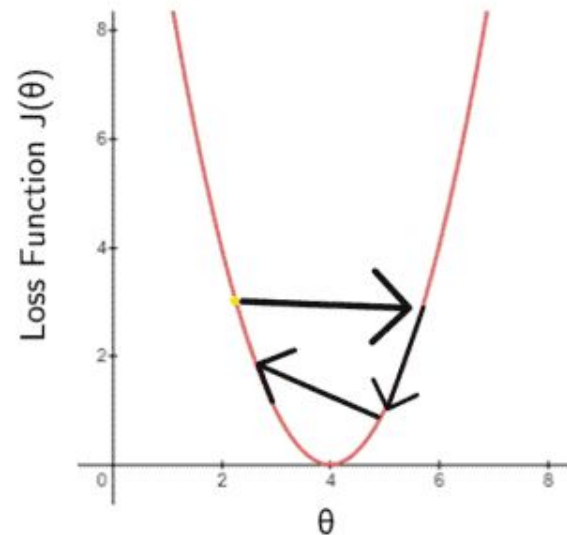
Small Learning Rate



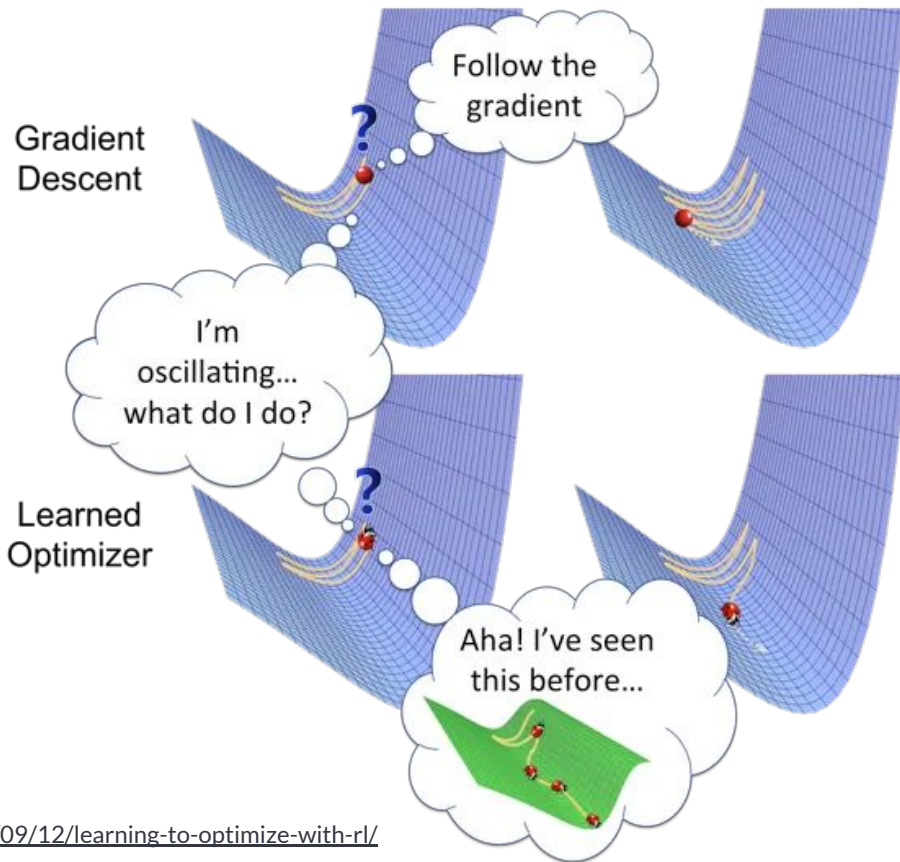
Optimal Scenario



Large Learning Rate



How to pick an optimizer!



Adaptive Learning Rate

Learning rates can be larger or smaller depending on

- How large the gradient is
- How fast learning is happening
- Size of particular weights
- etc

Adaptive Learning Rates

AdaGrad (2011)

Decreases the learning rate over time based on parameter updates.

AdaDelta (2012)

Improves AdaGrad by adapting learning rates without requiring a predefined learning rate. Accumulates past gradients to compute parameter updates dynamically.

RMSProp (2012)

Adapts learning rates per parameter using a running average of past gradients.

Adaptive Learning Rates

Adam (2015)

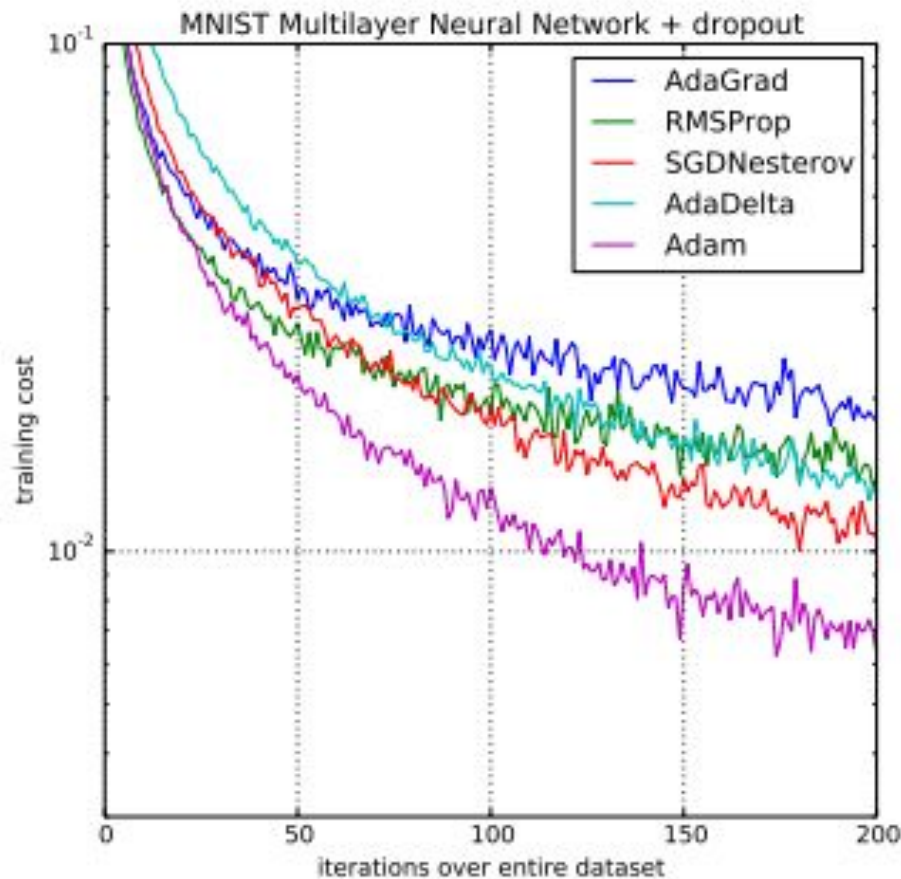
Combines RMSProp and momentum optimization. Tracks exponentially decaying averages of past gradients and squared gradients.

Standard optimizer for deep learning, requiring minimal tuning.

Nadam (2016)

Adam with Nesterov momentum for faster convergence.

Adaptive Learning Rates



General Recommendations:

- **Shallow models/smaller datasets:** RMSProp, AdaGrad.
- **Deep networks/large datasets:** Adam, Nadam.
- **Sequential data (e.g., time series, NLP):** RMSProp, Adam.
- **Computer vision tasks:** SGD with momentum, Adam.
- **Sparse features:** AdaGrad, AdaDelta.

(Shrestha & Mahmood, 2019)

Faster Optimizers Momentum

Momentum is a technique designed to accelerate gradient descent by “smoothing out” updates. In simple terms, momentum helps the optimization process gain velocity in directions of consistent descent, while damping oscillations in directions where the gradient sign keeps changing.

$$\begin{aligned} v^{(t+1)} &= \beta v^{(t)} - \eta \nabla_w \mathcal{L}(w^{(t)}), \\ w^{(t+1)} &= w^{(t)} + v^{(t+1)}. \end{aligned}$$

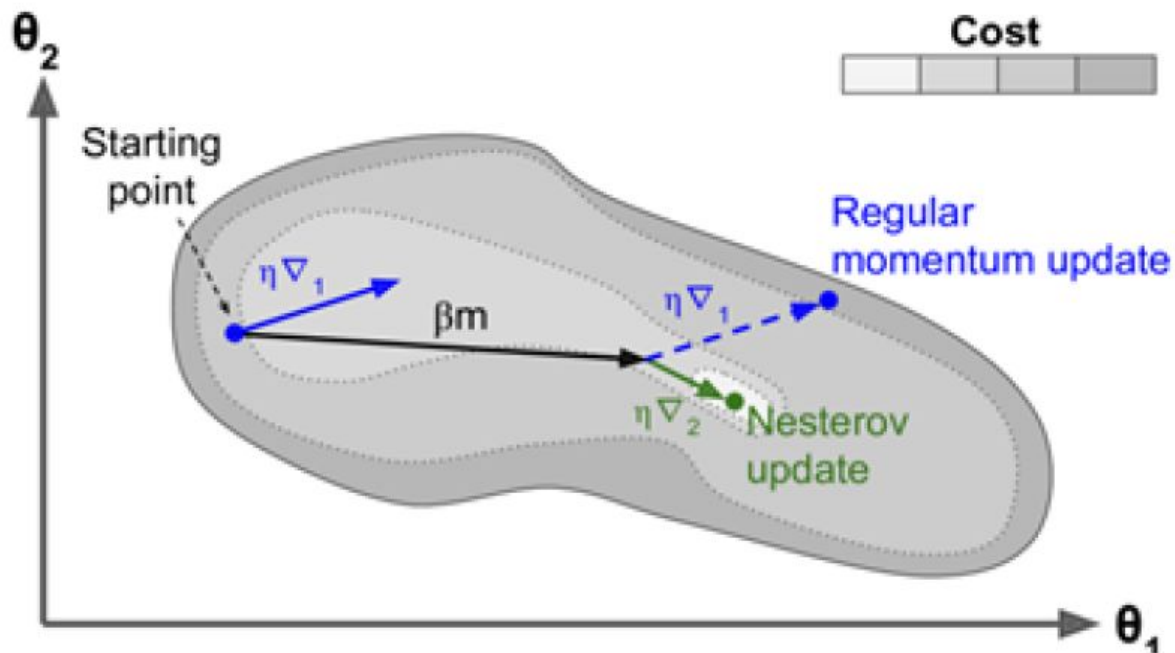
- w denote the parameter vector
- η denote the learning rate,
- β denote the momentum coefficient (usually in $[0,1]$),
- $\nabla_w \mathcal{L}$ denote the gradient of the loss $\mathcal{L}$ with respect to the parameters w

Nesterov Accelerated Gradient (NAG)

A related, often more effective variant is **Nesterov Accelerated Gradient**, which modifies how the gradient is calculated. The main difference is you look ahead in the parameter space before computing the gradient. The update rules become:

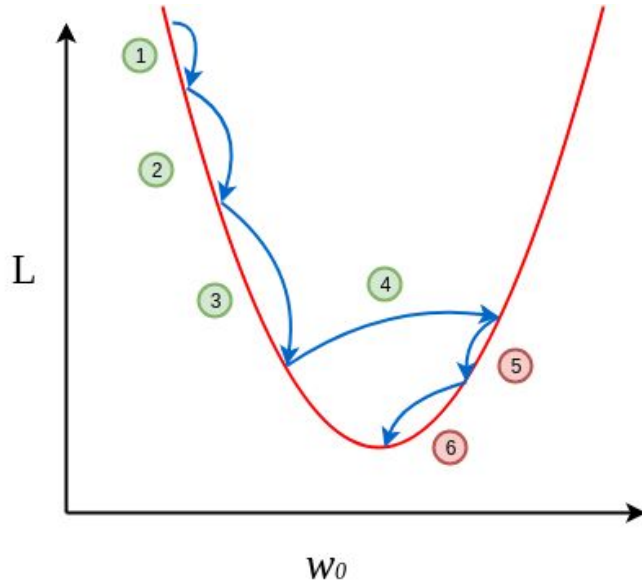
$$\begin{aligned} v^{(t+1)} &= \beta v^{(t)} - \eta \nabla_w \mathcal{L}(w^{(t)} - \beta v^{(t)}), \\ w^{(t+1)} &= w^{(t)} + v^{(t+1)}. \end{aligned}$$

Nesterov Accelerated Gradient (NAG)



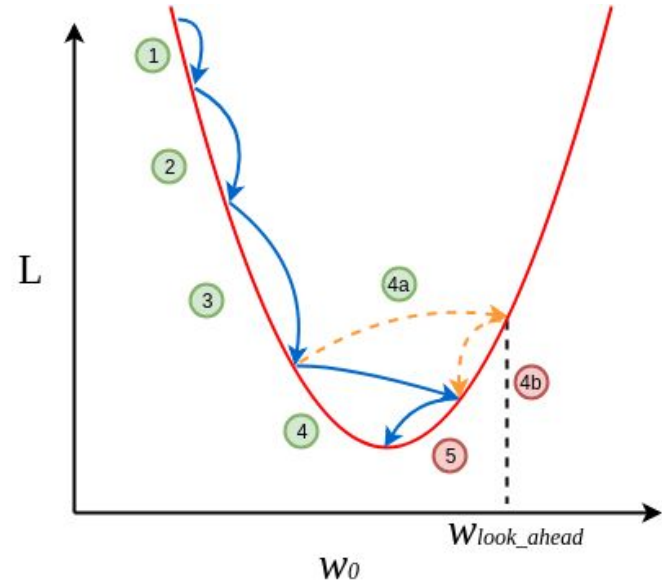
(Géron, 2019)

Momentum



(a) Momentum-Based Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$



(b) Nesterov Accelerated Gradient Descent

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

Optimizers Compared

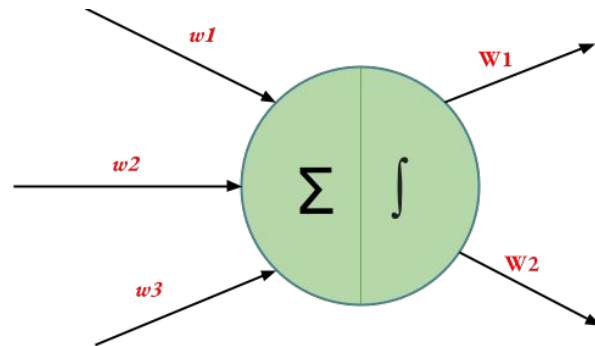
Optimizer	State Memory [bytes]	# of Tunable Parameters	Strengths	Weaknesses
SGD	0	1	Often best generalization (after extensive training)	Prone to saddle points or local minima Sensitive to initialization and choice of the learning rate α
SGD with Momentum	$4n$	2	Accelerates in directions of steady descent Overcomes weaknesses of simple SGD	Sensitive to initialization of the learning rate α and momentum β
AdaGrad	$\sim 4n$	1	Works well on data with sparse features Automatically decays learning rate	Generalizes worse, converges to sharp minima Gradients may vanish due to aggressive scaling
RMSprop	$\sim 4n$	3	Works well on data with sparse features Built in Momentum	Generalizes worse, converges to sharp minima
Adam	$\sim 8n$	3	Works well on data with sparse features Good default settings Automatically decays learning rate α	Generalizes worse, converges to sharp minima Requires a lot of memory for the state
AdamW	$\sim 8n$	3	Improves on Adam in terms of generalization Broader basin of optimal hyperparameters	Requires a lot of memory for the state
LARS	$\sim 4n$	3	Works well on large batches (up to 32k) Counteracts vanishing and exploding gradients Built in Momentum	Computing norm of gradient for each layer can be inefficient



Initialization and Activation Functions

Parameter Initialization

- **Constant Initialization**
 - All weights set to the same value (e.g., 0 or 0.01).
 - **Problem:** Neurons learn identical features → no useful training.
- **Random Initialization**
 - Weights sampled randomly **ignoring** layer size.
 - **Risk:** Can cause exploding or vanishing gradients in deep networks.
- **Xavier (Glorot) Initialization**
 - (for a normal distribution).
 - $\sigma^2 = \frac{2}{(n_{in} + n_{out})}$ balances variance for sigmoid/tanh activations.
- **He (Kaiming) Initialization**
 - $\sigma^2 = \frac{2}{n_{in}}$
 - **Advantage:** Ideal for ReLU (or similar) → keeps signals flowing.



fan_in = 3 (Number of input paths towards the neuron)

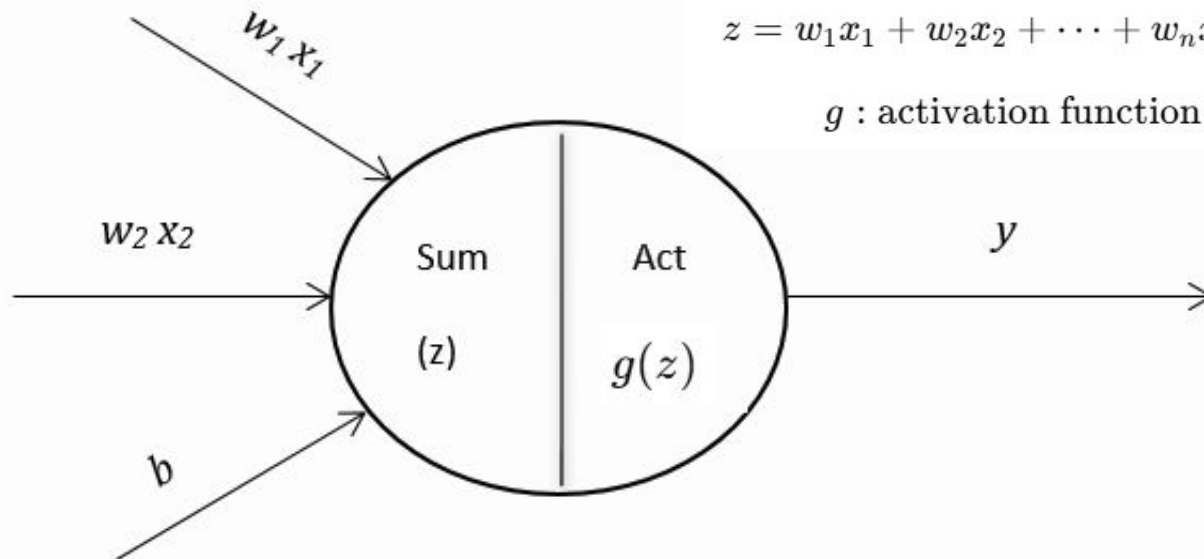
fan_out = 2 (Number of output paths towards the neuron)

Non saturating activation functions

$$y = g(z)$$

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

g : activation function



Why activation functions?

The need for these activation functions includes converting the linear input signals and models into non-linear output signals, which aids the learning of high order polynomials for deeper networks.

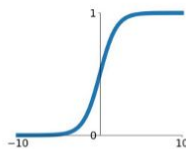
Non saturating activation functions

A "non-saturating activation function" is an activation function used in neural networks that does not reach a plateau or "saturate" at extreme input values

Activation Functions

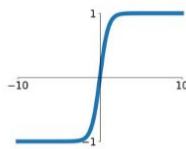
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



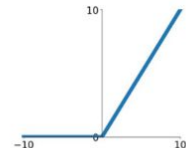
tanh

$$\tanh(x)$$



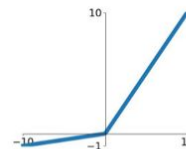
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

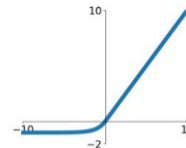


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



<https://medium.com/@shrutijadon/survey-on-activation-functions-for-deep-learning-9689331ba092>

Activation Functions - Comparison

Activation Function	Non-Saturating	Computational Cost	Key Benefit
ReLU	Yes	Low	Simplicity, efficiency
Leaky ReLU	Yes	Low	Prevents dying neurons
PReLU	Yes	Moderate	Adaptive learning
ELU	Yes	Moderate	Smooth negative values
SELU	Yes	Moderate	Self-normalizing properties
Swish	Yes	High	Improved gradient flow
Mish	Yes	High	Smoother, improved performance

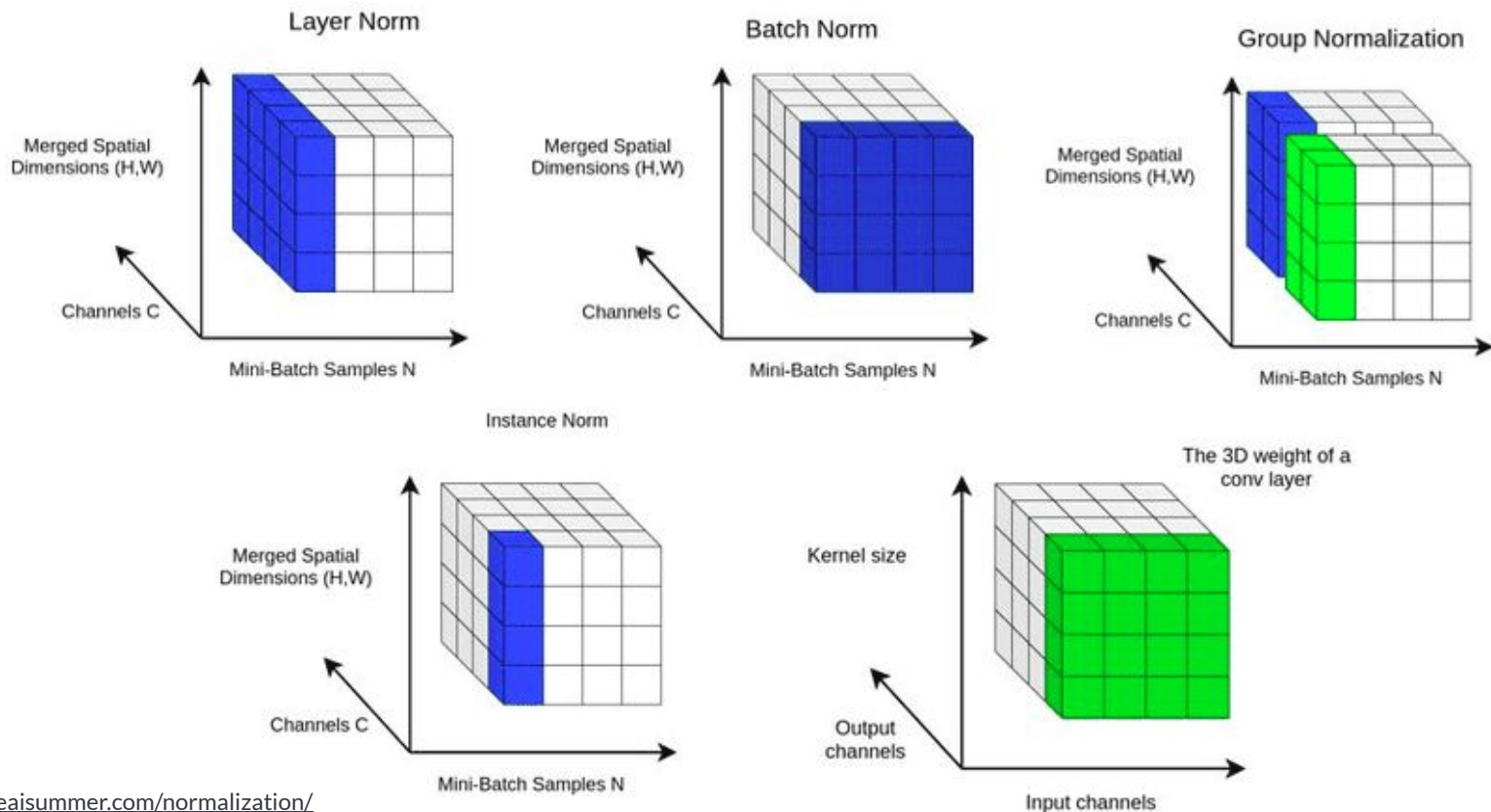
Choosing the Right Activation Function

- For general deep learning tasks: ReLU, Leaky ReLU.
- For deep models prone to vanishing gradients: ELU, SELU.
- For advanced models (e.g., transformers, vision tasks): Swish, Mish.
- For networks requiring adaptive behavior: PReLU.



Normalization

Normalization Approaches



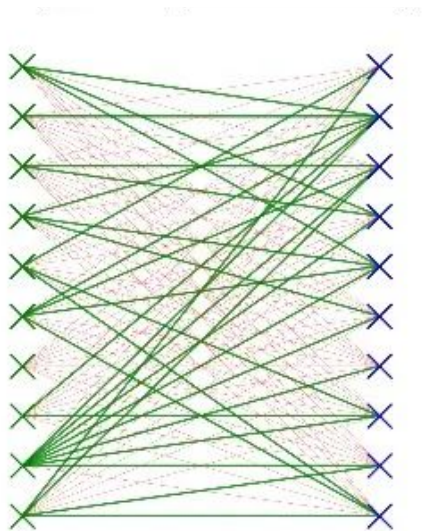
Normalization Approaches

Normalization Type	Normalization Scope	Best For
Batch	Across the batch, per feature	Deep CNNs with large batches
Layer	Across all features, per sample	RNNs, Transformers
Instance	Per channel, per instance	Style Transfer, Image Generation
Group	Within groups of channels, per instance	Object Detection, Small Batch Sizes



Overfitting

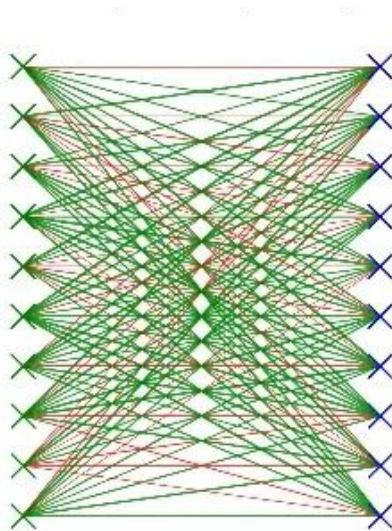
Regularization



Lasso

Sum of the absolute values of the weights.

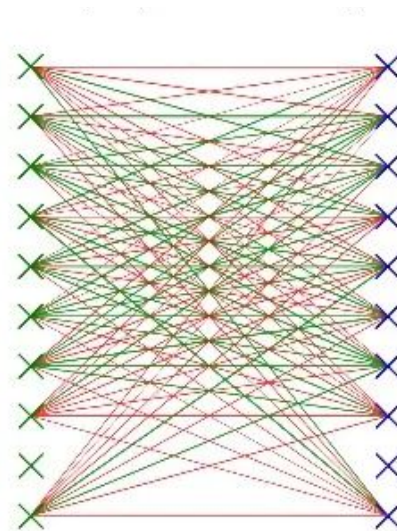
$$\lambda \sum |w|$$



Ridge

Sum of the squares of the weights.

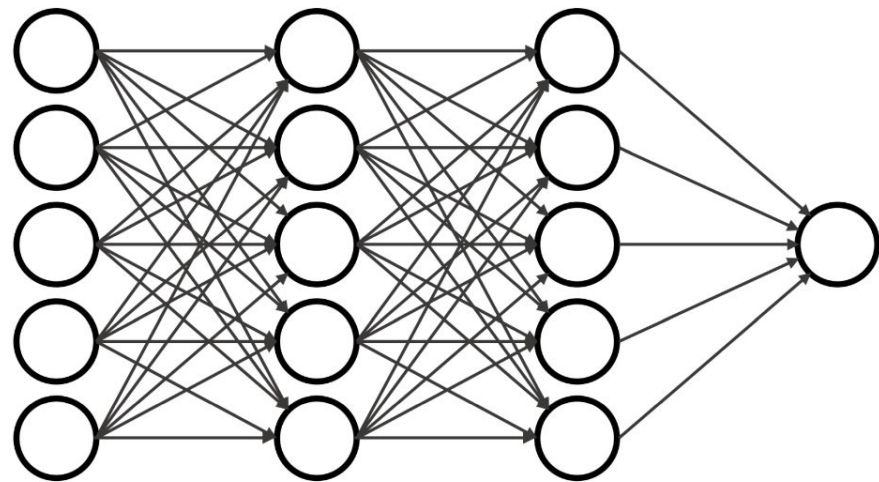
$$\lambda \sum w^2$$



Dropout

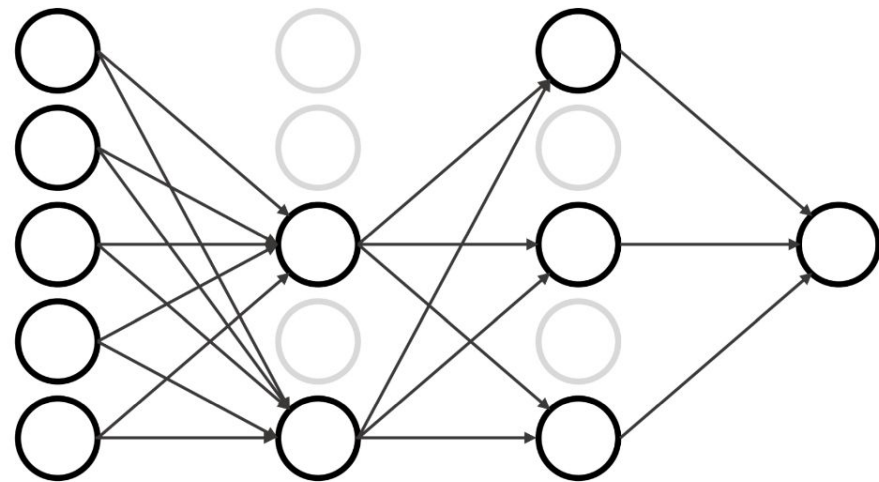
No explicit penalty term, but neurons are randomly “dropped” during training, forcing the network to rely on different sets of neurons in each iteration.

no dropout



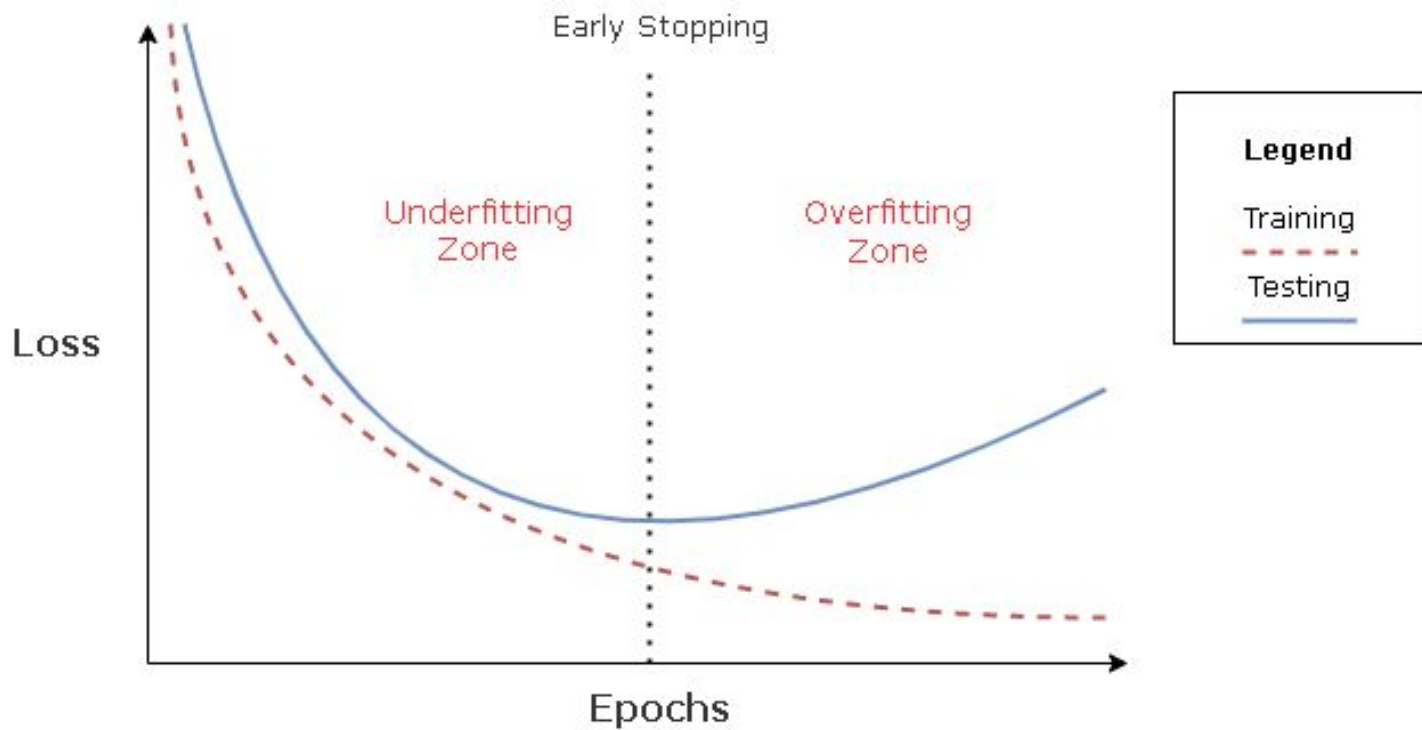
input dense layer dense layer

with dropout



input dense layer dense layer

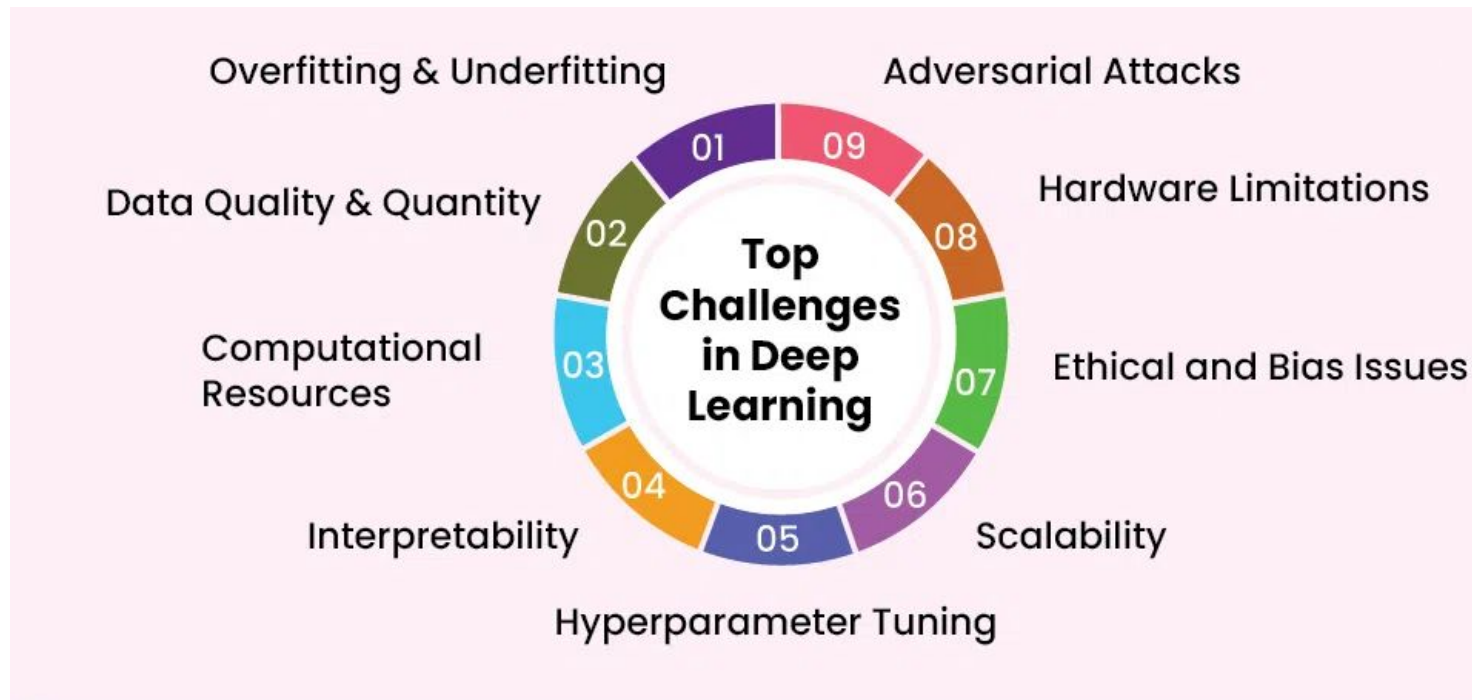
Early Stopping





Final Remarks

General Challenges in Advanced ML



Ways to Optimize Neural Network Training...Amplified!

1. Use efficient optimizers (e.g., AdamW, Adam, etc.).
2. Utilize hardware accelerators (GPUs/TPUs).
3. Max out the batch size.
4. Use Bayesian Optimization if the hyperparameter search space is large.
5. Set `max_workers` in the DataLoader.
6. Set `pin_memory` in the DataLoader.
7. Use mixed precision training.
8. Use He or Xavier initialization for faster convergence.
9. Use activation checkpointing to optimize memory (noting that runtime will increase).
10. Leverage multi-GPU training (Model/Data/Pipeline/Tensor parallelism).
11. For large models, use DeepSpeed, FSDP, or YaFSDP.
12. Normalize data after transferring to the GPU (especially for integer data).
13. Use gradient accumulation (though improvements may be marginal).
14. Always use `DistributedDataParallel`, not `DataParallel`.
15. Create tensors directly on the GPU (e.g., `torch.rand(2,2, device=...)`) rather than creating on CPU and then transferring (e.g., `torch.rand(2,2).cuda()`).



Available in our
repository!